UML

Formation NSI — Analyse Métier & Technique

Plan de la formation

1. **Introduction** — Pourquoi modéliser un système avec UML ?
2. **Cas d'Utilisation** (Use Case) — Périmètre et exigences fonctionnelles
3. **Activité** (Activity) — Flux de contrôle et d'opérations système
4. **Classes** (Class) — Concepts métier, attributs et relations
5. **État** (State Machine) — Cycle de vie d'un objet
6. **Séquence** (Sequence) — Chorégraphie des messages entre objets
7. **Méthode de modélisation** — Construire un modèle cohérent

Introduction

Qu'est-ce qu'UML ?

UML (Unified Modeling Language) est un langage visuel pour décrire les systèmes logiciels — leur structure, leur comportement et leurs interactions.

- *Pensez au **plan d'un architecte** : avant de construire un bâtiment, on le dessine avec une notation partagée que l'ingénieur, l'entrepreneur et le client savent tous lire sans ambiguïté.*

UML ne remplace pas le code : il rend l'**architecture invisible visible**, discutable et améliorable avant qu'aucune ligne ne soit écrite.

Définition - OMG UML 2.5

UML est un langage visuel de modélisation à usage général qui définit **14 types de diagrammes** organisés en deux familles : **structurels** (Structural) — la composition statique du système — et **comportementaux** (Behavioural) — la dynamique du système. Indépendant de toute technologie ou plateforme.

Pourquoi modéliser avec UML ?

Langage partagé *(Shared Language)*

Élimine les écarts de traduction entre analyste, développeur, testeur, manager.

Documentation d'architecture *(Architecture Doc)*

Rend l'architecture lisible sans plonger dans le code.

Validation des exigences *(Requirements Validation)*

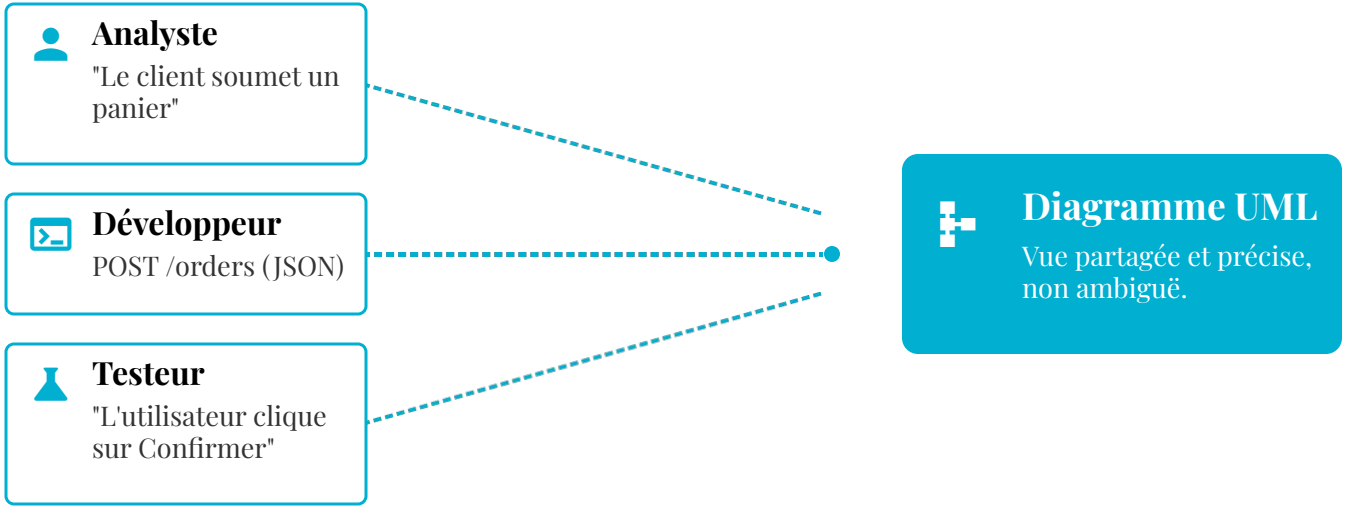
Détecte les lacunes avant le codage — 10× à 100× moins cher.

Transfert de connaissances *(Knowledge Transfer)*

Survit au départ des développeurs — connaissance organisationnelle.

> *"Le défaut le plus coûteux est celui découvert en production. Le moins cher est celui découvert sur un diagramme."*

Exemple : trois visions d'une même fonctionnalité



> *Trois vues valides mais partielles — chacun ne voit qu'une facette de la même fonctionnalité. Le diagramme UML les fait converger vers une spécification unique et non ambiguë.*

Pourquoi les organisations ne modélisent-elles pas ?

Trois objections fréquentes — trois mauvaises raisons :

- ***« Ça prend trop de temps »*** ↠ Un diagramme de 2h évite des jours de reprise. Le code livré « du premier coup » est ~16× plus rentable que le rework après revue de sprint.
- ***« Le code, c'est la documentation »*** ↠ Vrai au niveau micro, faux au niveau macro. Le code explique le comment ; UML explique le quoi et le pourquoi.
- ***« 14 diagrammes — c'est trop complexe »*** ↠ 4 types couvrent 90 % des besoins : Cas d'Utilisation, Activités, Classes, Séquence.

Piège — Vouloir tout apprendre d'un coup

Commencer par les 14 diagrammes = abandonner UML.

1. **Suivre la chaîne d'analyse** : Cas d'Utilisation ↠ Activité (avec couloirs Utilisateur / Système) ↠ Classes.
2. Ajouter Séquence pour les **interactions externes** et État pour les **cycles de vie** — pas avant.

Le paysage des diagrammes UML 2.5

Comportementaux — la dynamique

Diagramme	Ce qu'il montre
Cas d'Utilisation ✓	Acteurs, fonctionnalités, périmètre
Activité ✓	Flux de contrôle et de données
État / Séquence ✓	Cycle de vie & échanges temporels

Structurels — l'anatomie

Diagramme	Ce qu'il montre
Classes (Class) ✓	Concepts métier, attributs et relations
Composants	Système, interfaces, dépendances
Déploiement...	Variantes spécialisées (Objets, Paquetages)

✓ = couverts dans cette formation.

Points clés — Introduction

- **UML** (Unified Modeling Language) est un langage visuel normalisé pour spécifier la structure et le comportement d'un système — il ne remplace pas le code, il fait le pont entre métier et technique.
- UML crée un **langage partagé** lisible par analystes, développeurs, testeurs et managers — la validation devient possible avant le codage.
- Les trois objections classiques (temps, code auto-documenté, complexité) sont en réalité des **arguments en faveur** de la modélisation.
- UML 2.5 = **14 types de diagrammes** ; la plupart des projets n'en utilisent que 3 ou 4.
- **Suivre la chaîne d'analyse** : BPMN ↠ Cas d'Utilisation ↠ Activité (couloirs Utilisateur / Système) ↠ Classes. Action côté Utilisateur = un écran ; action côté Système = une méthode.

Cas d'Utilisation

Cas d'Utilisation — Définition

Un **diagramme de cas d'utilisation** (Use Case Diagram) répond à la question : que doit faire ce système, et pour qui ?

- Cartographie l'ensemble du **périmètre fonctionnel** sur une page
- Le seul diagramme qu'une partie prenante métier peut lire et valider sans formation technique
- Point d'entrée standard pour la **collecte des exigences**

Définition - OMG UML 2.5

Diagramme comportemental qui capture les exigences fonctionnelles en représentant les interactions entre des entités externes (les **acteurs** — Actors) et les fonctions observables du système (les **cas d'utilisation** — Use Cases), à l'intérieur d'une **frontière de système** (System Boundary) explicite.

Acteur (Actor)

Toute entité **extérieure au système** qui interagit avec lui — représente un **rôle**, jamais une personne.

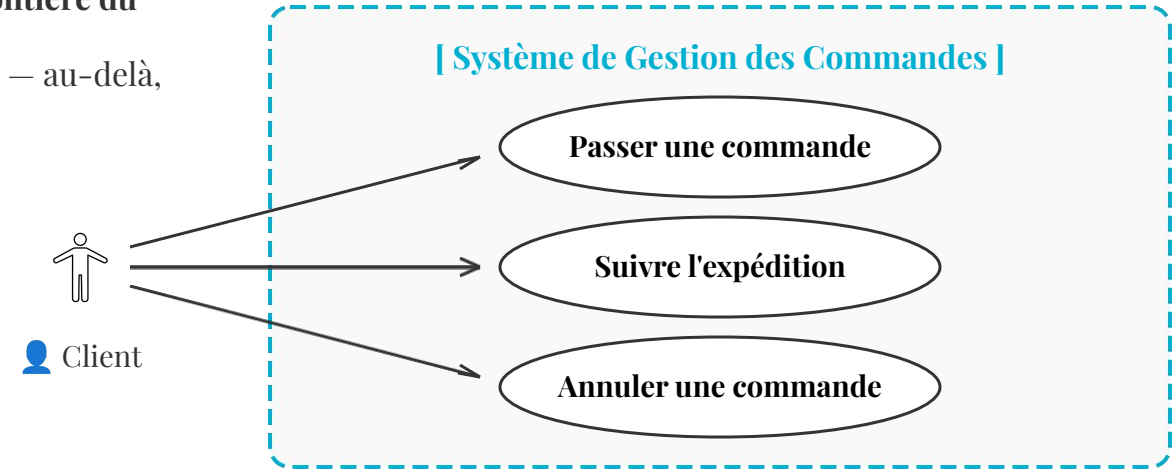
Type	Position	Rôle
Principal	À gauche	**Initie** l'interaction
Secondaire	À droite	**Soutient** le système ou **reçoit** un résultat

---

💡 *Oublier les acteurs secondaires = source classique d'**exigences d'intégration manquantes**.*

Un **cas d'utilisation** est une fonction **unique et observable** fournie par le système pour satisfaire l'objectif d'un acteur.

- Forme : **ovale** étiqueté **verbe** + **objet** (« Passer une commande »)
- Vit **à l'intérieur** du rectangle de la **frontière du système** (System Boundary)
- 5 à 15 cas d'utilisation par diagramme — au-delà, redécomposer



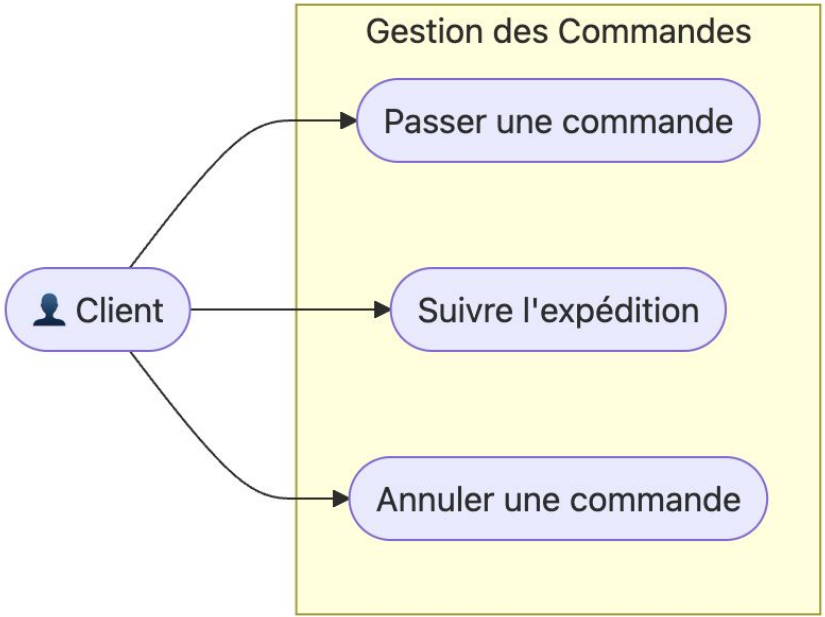
💡 *Un cas d'utilisation = un **objectif métier**, pas une action système (« Valider l'entrée », « Insérer en base »).*

Cas d'Utilisation (Use Case) & Frontière

Un cas d'utilisation (Use Case) est une fonction **unique et observable** que le système fournit pour satisfaire l'objectif d'un acteur.

- Forme : **ovale** étiqueté **verbe** + **objet** (« Passer une commande »)
- Vit à **l'intérieur** du rectangle de la **frontière du système** (System Boundary)
- **5 à 15 cas d'utilisation par diagramme** — au-delà, re-décomposer

Un cas d'utilisation = un **objectif métier, pas une action système !**



Relations — Association & Inclusion

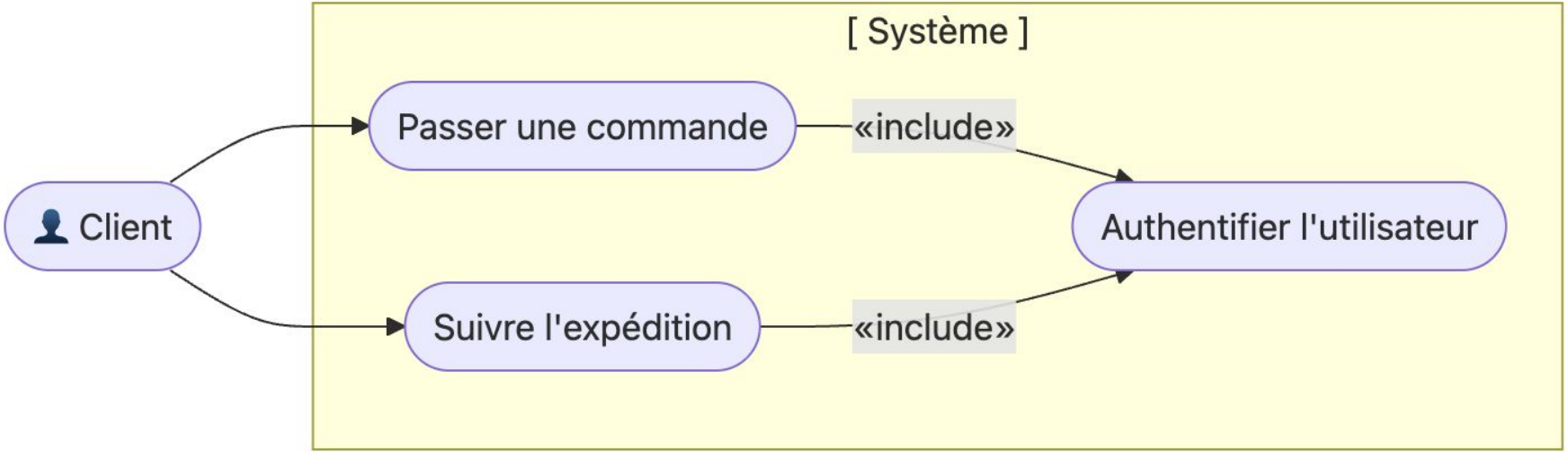
Association (Association)

- Trait plein acteur ↔ cas d'utilisation : « cet acteur participe à ce cas d'utilisation ».

Inclusion (Include)

- Flèche pointillée <<include>> du cas de base vers le cas inclus — comportement partagé obligatoire (le base appelle toujours l'inclus).

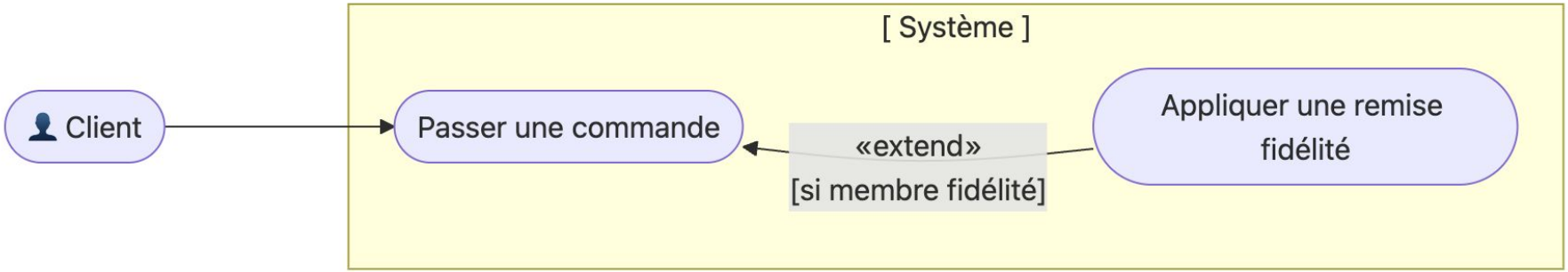
Relations — Association & Inclusion



Motif classique : factoriser l'authentification partagée par plusieurs cas.

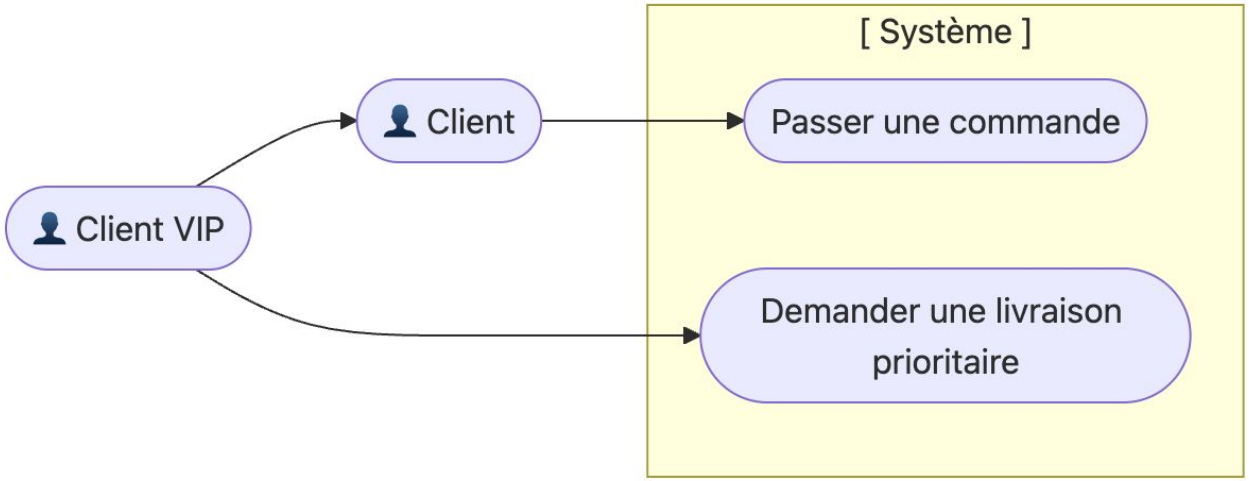
Extension (Extend)

Flèche pointillée <<extend>> de l'extension **vers** le cas de base — **comportement optionnel et conditionnel**.



Généralisation (Generalization)

Flèche pleine à pointe triangulaire creuse — héritage entre acteurs ou entre cas d'utilisation. Un « Client Premium » hérite de tous les cas d'un « Client ».



Include vs Extend — le piège classique

- **Include** = la flèche **sort** du cas de base (le base appelle toujours).
- **Extend** = la flèche **entre** dans le cas de base (l'extension se branche **conditionnellement**).
- Test : « Le cas de base se termine-t-il sans ce comportement ? » Oui ↠ Extend ; Non ↠ Include.

Description Textuelle (Textual Description)

Le diagramme est la **carte** ; la description textuelle est le **territoire**.

MODÈLE STANDARD (UC-XX)

Nom : Verbe + Objet
Acteur(s) : Principal / Secondaires
Préconditions : Ce qui doit être vrai
Postconditions : Ce qui est garanti

Flux Principal : Étapes numérotées
Flux Alternatif A : Déviation à l'étape N
Flux Exception E : Erreur à l'étape N

HAPPY PATH
Le **flux principal** représente le chemin nominal sans encombre.

DÉVIATIONS
Les **flux alternatifs** sont les déviations attendues du système.

ERREURS
Les **flux d'exception** gèrent les conditions d'erreur critiques.

Description Textuelle (Textual Description) - Exemple

Champ	Contenu
Identifiant	UC-03
Nom	Passer une commande
Acteur Principal	Client
Préconditions	Le Client est authentifié. Au moins un produit existe dans le panier du client. Le Client possède une adresse de livraison enregistrée.
Postcondition — Succès	La commande est créée avec le statut `Confirmée`. Le stock est réservé. Un courriel de confirmation est envoyé au client.
Postcondition — Échec	Aucune commande n'est créée. Le contenu du panier est préservé pour la prochaine session du client.

Flux Principal

1. Le Client examine le contenu et les quantités du panier.
2. Le Client sélectionne ou confirme l'adresse de livraison.
3. Le Client clique sur « Confirmer la commande ».
4. Le système vérifie que tous les articles du panier disposent d'un stock suffisant.
5. Le système vérifie que le total de la commande ne dépasse pas la limite de crédit du client.
6. Le système crée la commande avec le statut `Confirmée`.
7. Le système réserve le stock pour chaque ligne de commande.
8. Le système envoie un courriel de confirmation au client.
9. Le système affiche la page de confirmation de commande avec le numéro de référence.

Flux Alternatif A — Rupture de stock (déviation à l'étape 4)
4a. Un ou plusieurs articles ne disposent pas d'un stock suffisant.
4b. Le système affiche un message de disponibilité listant les articles concernés et leur stock actuel.
4c. Le cas d'utilisation se termine. Aucune commande n'est créée.

Flux Alternatif B — Limite de crédit dépassée (déviation à l'étape 5)
5a. Le total de la commande dépasse la limite de crédit approuvée du client.
5b. Le système affiche un avertissement de crédit montrant la limite et le total de la commande.
5c. Le cas d'utilisation se termine. Aucune commande n'est créée.

Pièges courants — Cas d'Utilisation

- **Cas nommé comme une fonction système** (« Valider l'entrée ») au lieu d'un objectif d'acteur (« Passer une commande »).
- **Trop de cas sur un diagramme** (>15) ↠ catalogue illisible. Découper en domaines fonctionnels.
- **Sens des flèches Include/Extend inversé** ↠ modèle syntaxiquement plausible, sémantiquement faux.
- **Sauter la description textuelle** ↠ le diagramme nomme les fonctionnalités, la description les spécifie. Les deux sont requis avant développement.

Points clés — Cas d'Utilisation

- Le diagramme cartographie **qui** utilise le système (acteurs) et **ce qu'il fait pour eux** (cas d'utilisation), à l'intérieur d'une frontière de système explicite.
- **Acteurs principaux** initient (à gauche) ; **acteurs secondaires** soutiennent ou reçoivent (à droite). Toujours un rôle, jamais une personne.
- **Include** (<<include>>) = comportement partagé obligatoire ; **Extend** (<<extend>>) = comportement optionnel sous condition.
- **Généralisation** = héritage de rôle entre acteurs ou spécialisation entre cas d'utilisation.
- Chaque cas non trivial requiert une **description textuelle** : préconditions, flux principal, flux alternatifs, flux d'exception, postconditions.

Activité

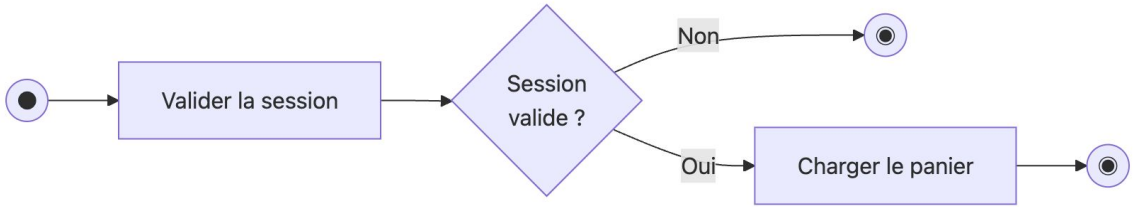
Activité — Définition

Un **diagramme d'activité** (Activity Diagram) modélise comment une opération s'exécute, étape par étape, y compris les décisions, le travail parallèle et les données qui circulent.

Dimension	**BPMN**	**Activité UML**
Audience	Métier	Développeurs, architectes
Modélise	Processus métier inter-organisationnels	Opérations systeme, algorithmes
Idéal pour	“Que fait ce métier ?”	“Comment ce système fonctionne?”

Nœud Initial & Nœuds Finaux

Nœud	Forme	Rôle
Initial *(Initial Node)*	●	Point de départ unique
Final d'Activité *(Activity Final)*	◉	**Termine toute l'activité** (équiv. Terminate End BPMN)
Final de Flux *(Flow Final)*	⊗	Termine **uniquement le flux courant**



⚠️ *Sur une branche parallèle qui doit juste se terminer, utiliser Final de Flux ⊗ — sinon ◉ tue tous les autres flux.*

Action & Flux de Contrôle

Action *(Action)*

Action Name

- **Rectangle aux coins arrondis** (identique BPMN)
- Étape atomique nommée **verbe** + **objet**
- Pas de sous-types au niveau diagramme

Flux de Contrôle *(Control Flow)*

`[condition]`

- **Flèche pleine** transportant un jeton
- Peut porter une **garde** `[condition]` entre crochets

Décision & Fusion

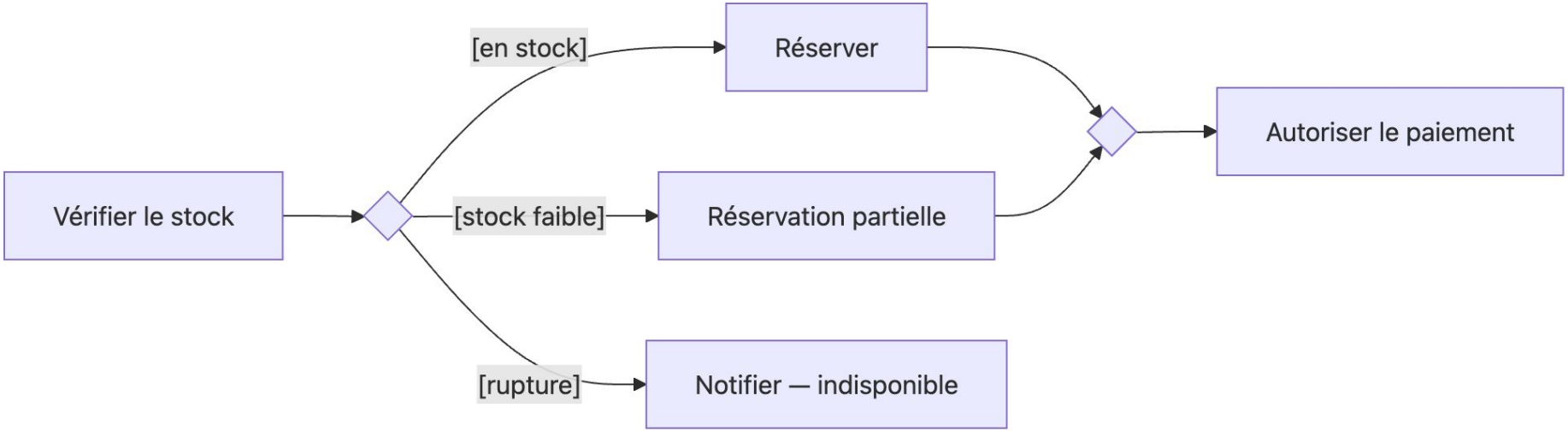
Nœud de Décision (Decision Node)

- Losange creux, 1 entrée ↠ N sorties gardées. Équivalent UML d'une passerelle exclusive BPMN. Toujours prévoir une garde [sinon].

Nœud de Fusion (Merge Node)

- Losange creux, N entrées ↠ 1 sortie sans garde. Laisse passer le premier jeton arrivé sans attendre les autres.

Décision & Fusion



Fourche & Jointure (Fork & Join)

Fourche (Fork Node)

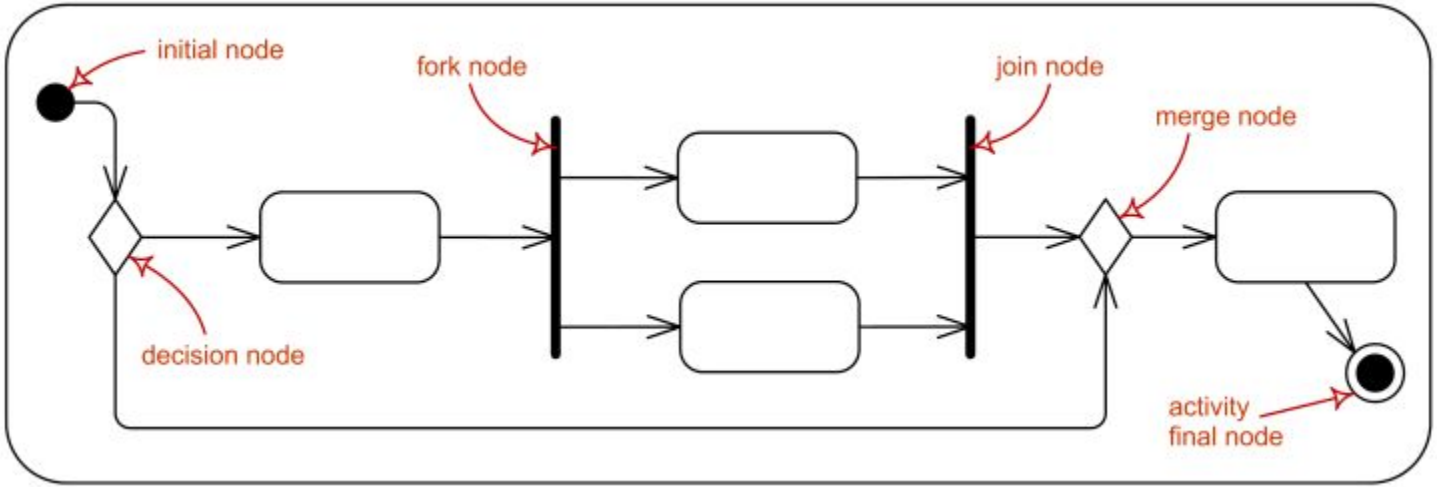
- Barre épaisse : 1 entrée ↠ N sorties simultanées sans condition. Équivalent UML d'une passerelle parallèle BPMN.

Jointure (Join Node)

- Barre épaisse : N entrées ↠ 1 sortie. Attend tous les jetons — barrière de synchronisation stricte.

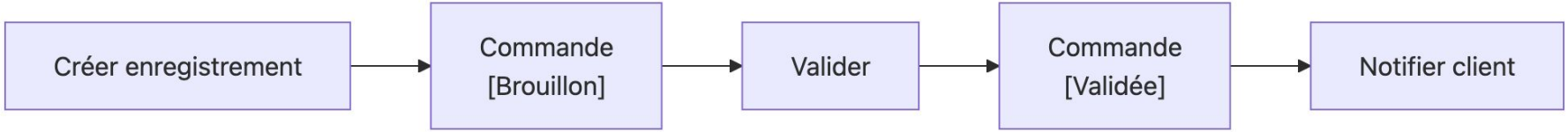
⚠️ Si une branche parallèle se bloque, la jointure attend indéfiniment. Chaque branche doit avoir un chemin de complétion.

Fourche & Jointure (Fork & Join)

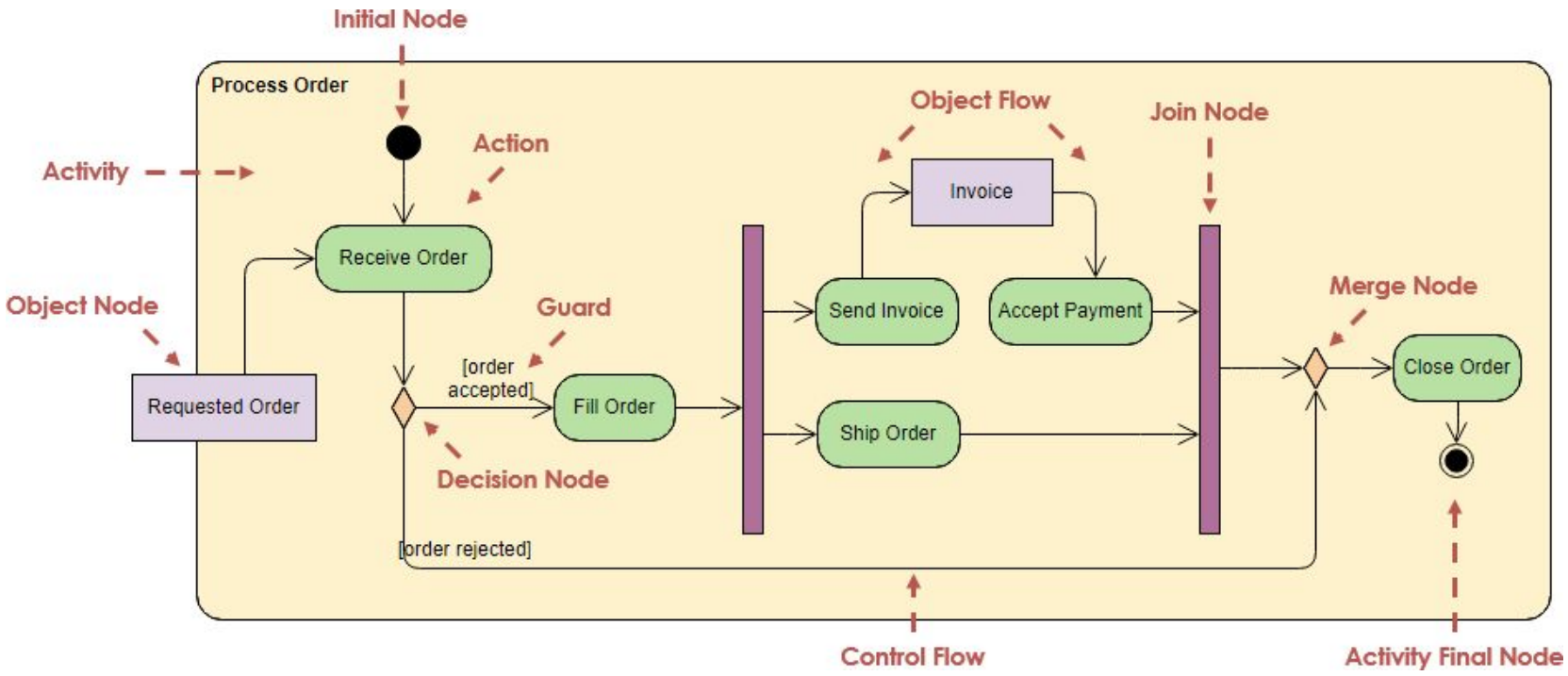


Nœud d'Objet (Object Node)

- **Rectangle simple** (sans coins arrondis)
- Représente une **donnée** circulant entre actions, étiquetée avec son **état** : [Brouillon], [Validée]
- Équivalent UML des **objets de données** (Data Objects) BPMN
- Chaque nœud d'objet correspond à un **attribut** du Diagramme de Classes ; les états annotés correspondent aux valeurs prises par cet attribut au fil du flux



Nœud d'Objet (Object Node)

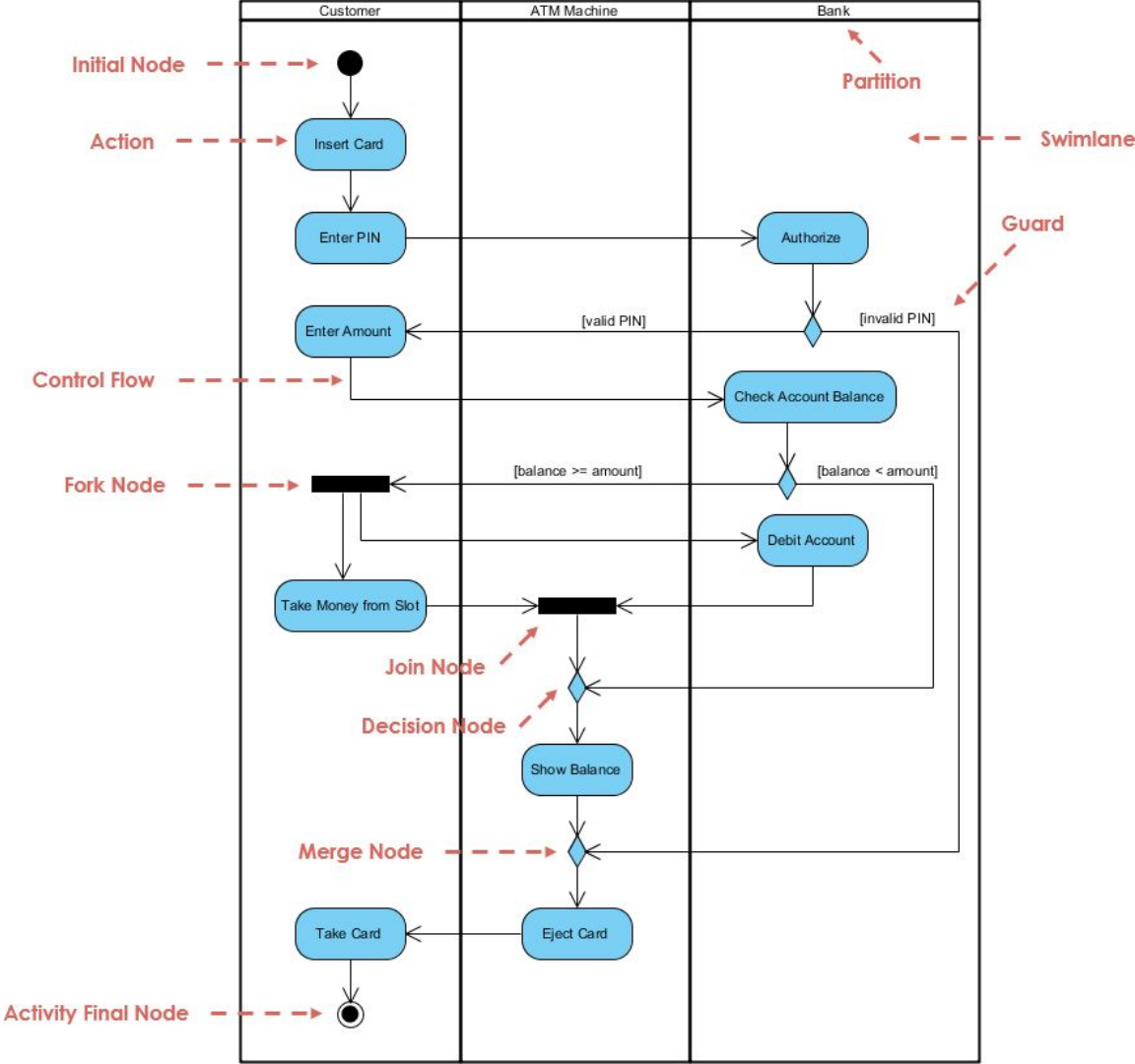


Partition (Partition / Swimlane)

Pour la spécification d'un Système d'Information, on adopte systématiquement :

- un couloir par acteur humain (frontend) — saisir, cliquer, consulter à l'écran
- un couloir unique Système d'Information (backend) — valider, calculer, persister, appeler une API, envoyer un courriel
- optionnellement, un couloir par acteur secondaire système — passerelle de paiement, ERP, autre SI

Partition (Partition / Swimlane)



La règle de dérivation Activité → Classes

Le diagramme d'Activité fait émerger le diagramme de Classes. La convention de partitionnement matérialise cette dérivation :

Élément du diagramme d'Activité	**Devient dans le système**
Action côté **Utilisateur** (couloir Utilisateur)	Un **écran** (screen) de l'application
Action côté **Système** (couloir SI)	Une **méthode** (method) du Diagramme de Classes
Nœud d'Objet circulant entre actions	Un **attribut** (attribute) du Diagramme de Classes

C'est cette règle qui assure la **traçabilité** du processus métier jusqu'aux méthodes et aux attributs des classes — l'objectif pédagogique central.

Distinction avec BPMN

- En **BPMN**, chaque équipe ou rôle métier a son couloir ↠ modélisation **multi-acteurs centrée processus**.
- En **UML Activity**, le SI est isolé dans son propre couloir ↠ modélisation **centrée système** qui préfigure la conception logicielle.

Pour un processus métier multi-acteurs sans focus système, BPMN reste le bon outil.

Pièges courants — Activité

- **Final d'Activité ◉ sur une branche parallèle** ↠ tue toute l'activité, pas seulement la branche. Utiliser **Final de Flux ⊗**.
- **Confondre Décision/Fusion et Fourche/Jointure** ↠ Décision = 1 chemin sur N (exclusif) ; Fourche = N chemins simultanés (parallèle).
- **Garde manquante** sur une arête de Décision ↠ se déclenche toujours, comportement de fourche inconditionnelle. Toujours une [sinon].
- **Action sans flux sortant** ↠ jeton suspendu indéfiniment. Chaque chemin doit atteindre un nœud final.

Points clés — Activité

- Un diagramme d'activité UML spécifie **comment** chaque Cas d'Utilisation s'exécute. C'est le **maillon central** de la chaîne UC ↠ Activité ↠ Classes.
- **Règle de dérivation** : action **Utilisateur** = un **écran** ; action **Système** = une **méthode** des Classes ; nœud d'objet = un **attribut**. Les couloirs ne sont pas optionnels — ils matérialisent cette règle.

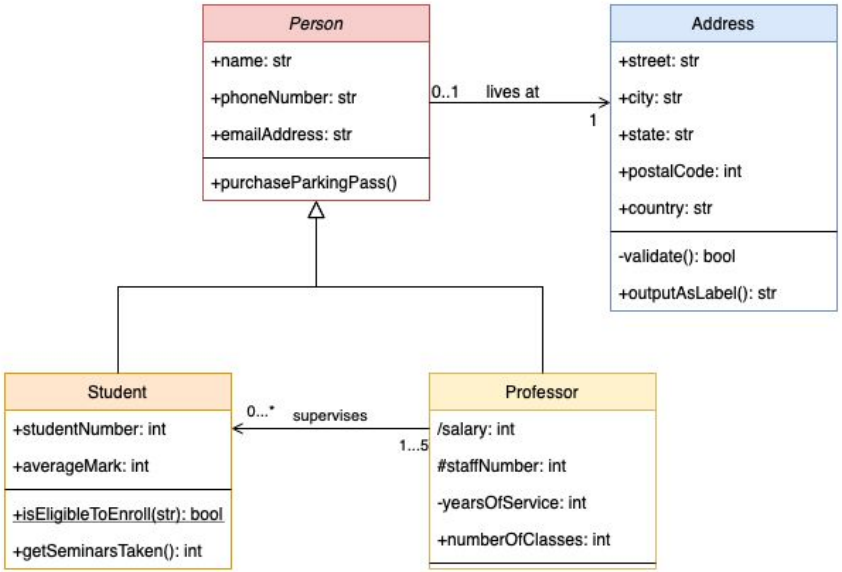
Points clés — Activité

- **Actions** (rectangles arrondis) + **flux de contrôle** (flèches gardées) + **nœuds d'objet** (rectangles simples avec état).
- **Décision/Fusion** (losanges) = routage exclusif d'un seul jeton ; **Fourche/Jointure** (barres) = parallélisme et synchronisation.
- **Convention de partitionnement** : un couloir par **acteur humain** (frontend) + un couloir **Système d'Information** (backend), parfois un couloir par acteur secondaire système.
- **Final d'Activité** ◉ termine tout ; **Final de Flux** ⊗ termine une seule branche.

Classes

Classe — Définition

Le diagramme de classes est **l'aboutissement structurel de la chaîne d'analyse** — il répond à : *quels concepts métier le système gère, à quoi ressemblent-ils et comment se relient-ils ?*



Définition - OMG UML 2.5

Diagramme structurel qui modélise la structure statique du système en représentant ses **classes** (Classes), leurs **attributs** (Attributes), leurs **opérations** (Operations) et les relations entre elles. Sert simultanément les **analystes** (modèle de domaine), les **architectes de données** (schéma BDD) et les **développeurs** (code OO).

Les méthodes émergent de l'Activité

Le diagramme de Classes ne s'invente pas. Pour chaque cas d'utilisation:

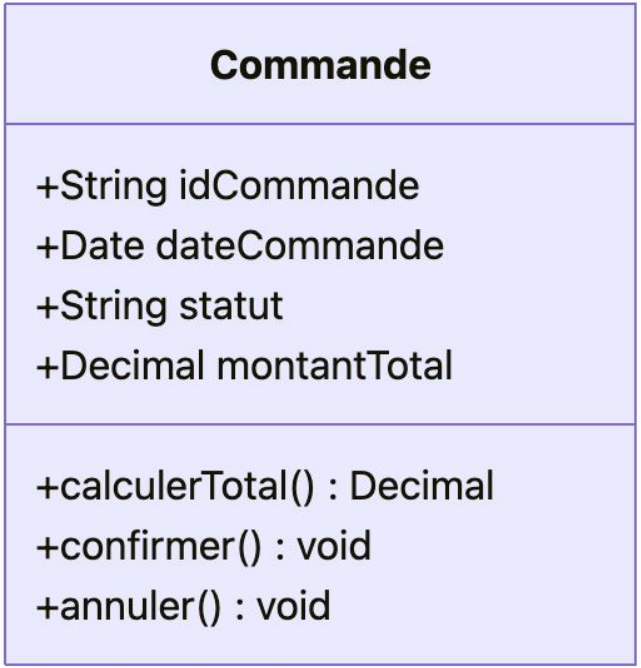
- les actions du couloir Système du diagramme d'Activité **deviennent des méthodes**.
- les objets qui circulent entre ces actions **deviennent des attributs**.

Le travail consiste à regrouper les méthodes par classe et à formaliser les attributs.

Anatomie d'une Classe

Rectangle à trois compartiments : nom (haut), attributs (milieu), opérations (bas).

1. **Nom** en PascalCase, nom commun au singulier : Commande, Client, Facture
2. **Attribut** : visibilité nom : Type [= valeurParDéfaut]
3. **Opération** : visibilité nom(param: Type) : TypeRetour — verbe décrivant un comportement métier



Visibilité (Visibility)

Définit **qui peut accéder** à un membre d'une classe.

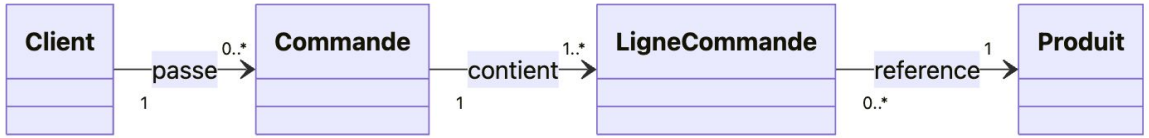
Symbole	Nom	Signification
+	**Public**	Accessible de partout
–	**Privé**	Classe elle-même seulement
#	**Protégé**	Classe + sous-classes
~	**Paquetage**	Classes du même paquetage

↠ *Au niveau **analyse**, la plupart des attributs sont **+** (modèle de données).*
↠ *Au niveau **conception**, on utilise l'encapsulation : attributs **–** et opérations **+**.*

Multiplicité (Multiplicity)

Aux **deux extrémités** d'une association : combien d'instances participent.

Notation	Signification
`1`	Exactement un
`0..1`	Zéro ou un (optionnel)
`*`	Zéro ou plusieurs
`1..*`	Un ou plusieurs
`2..5`	Entre 2 et 5



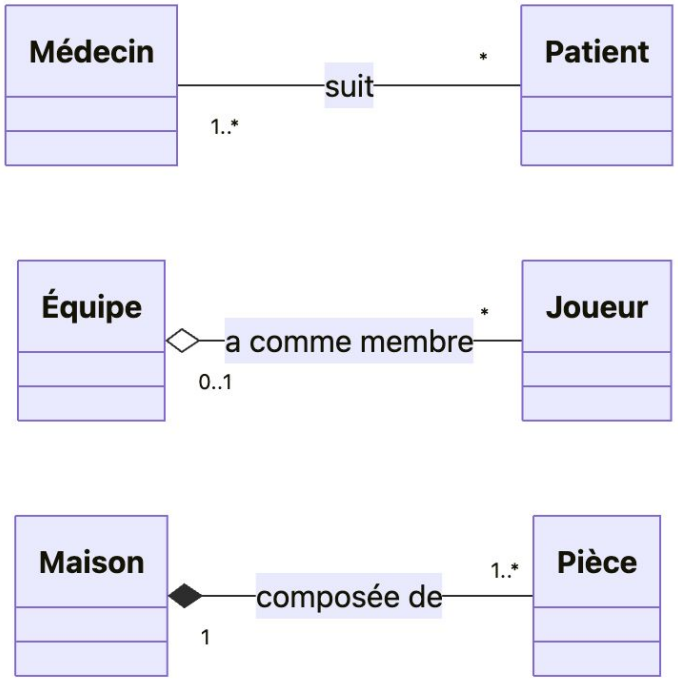
> *La multiplicité encode des **règles métier** : `1` vs `0..1` détermine la **nullabilité**.*
> *`1` vs `1..*` détermine si une entité peut exister sans enfants (ex: Commande sans lignes).*

Relations — Association, Agrégation, Composition

Relation	Notation	Sémantique
Association	⟶	Connexion structurelle « a un »
Agrégation (Aggregation)	◇—	Tout/partie **faible** — la partie survit au tout
Composition	◆—	Tout/partie **forte** — la partie meurt avec le tout

> *La composition implique une responsabilité du cycle de vie (destruction en cascade).*

Relations — Association, Agrégation, Composition



Relation	Exemple concret	Que se passe-t-il si on supprime le « tout » ?
Association (trait plein)	Médecin — Patient	Aucun n'est le « tout » — **les deux existent indépendamment**
Agrégation (◇ losange creux côté tout)	Équipe ◇— Joueur	Le joueur **continue d'exister** (peut changer d'équipe ou rester libre)
Composition (◆ losange plein côté tout)	Maison ◆— Pièce	La pièce **disparaît avec** la maison — sans maison, le concept de pièce n'a plus de sens

Le test du « si je supprime »

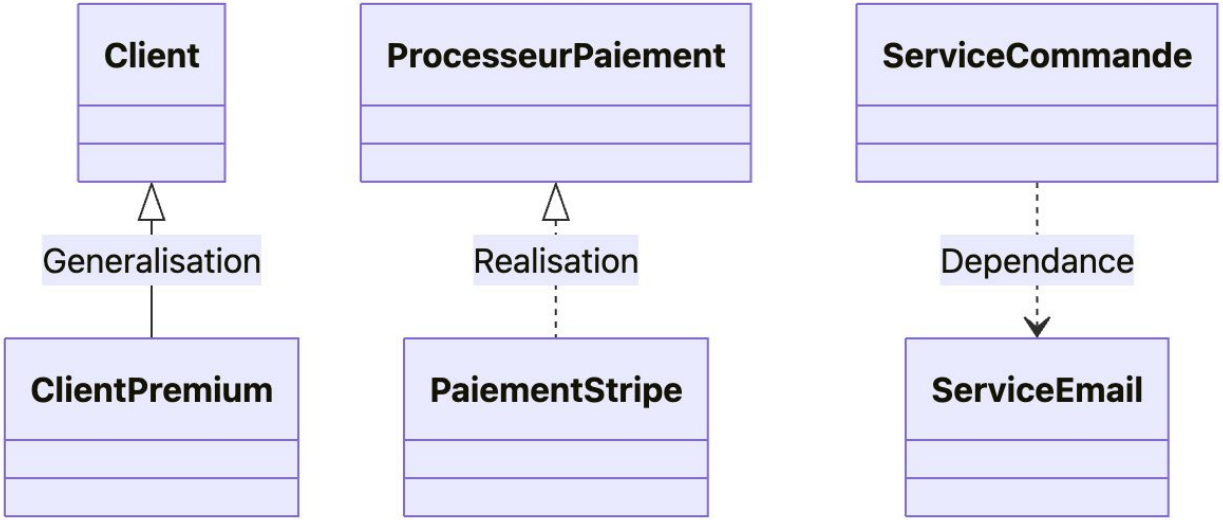
- Patient sans Médecin ↠ toujours une personne ✅ ↠ **Association**
- Joueur sans Équipe ↠ libre, peut être recruté ailleurs ✅ ↠ **Agrégation**
- Pièce sans Maison ↠ absurde, n'existe plus ❌ ↠ **Composition**

En cas de doute pour le métier : presque toutes les relations « contient » sont des compositions (ex. Commande ◆— LigneCommande : supprimer la commande supprime ses lignes — une LigneCommande orpheline n'a aucun sens).

Relations — Généralisation, Réalisation, Dépendance

Relation	Notation	Sémantique
Généralisation *(Generalization)*	◁————	« est un » — héritage (trait plein, triangle **creux**)
Réalisation *(Realization)*	◁- - - - - - - -	« implémente » — contrat d'**interface** (pointillé, triangle creux)
Dépendance *(Dependency)*	<- - - - - - - - - - - -	« utilise temporairement » — pas de référence permanente (flèche pointillée)

Relations — Généralisation, Réalisation, Dépendance



Test « est-un » vs « a-un » : **est-un** ↠ **Généralisation** ; **a-un** ↠ **Association**.
Une **interface** (stéréotype <<interface>>) ne déclare que des signatures, sans implémentation.

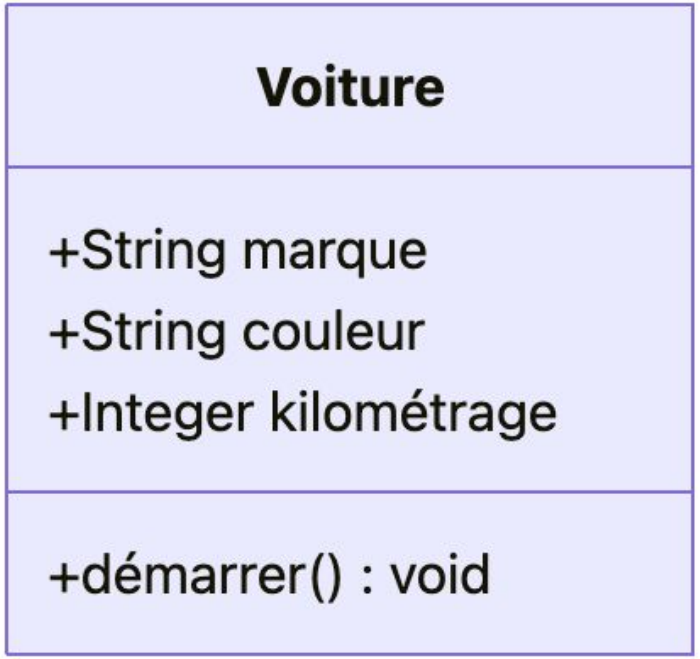
Classe vs Instance — vocabulaire essentiel

Terme	Définition	Analogie du quotidien
Classe *(Class)*	Le **modèle** qui décrit ce qu'est une catégorie d'objets — ses attributs, ses méthodes	Le **plan d'architecte** d'une maison
Instance *(Instance)*	Un **exemplaire concret** créé à partir du modèle, avec ses **propres valeurs**	La **maison construite** au 12 rue des Lilas
Instancier *(Instantiate)*	Le verbe = **créer** une nouvelle instance à partir d'une classe	**Construire** une maison à partir du plan

D'autres analogies utiles

- **Recette de gâteau** (classe) → **chaque gâteau sorti du four** (instance)
- **Modèle de CV** vierge (classe) → **CV de Marie**, **CV de Paul** remplis (instances)
- **Formulaire de déclaration** vide (classe) → **chaque déclaration soumise** (instance)

Classe vs Instance — vocabulaire essentiel



À partir de cette classe Voiture, on instancie des objets concrets — par exemple :

- *voiture1 : Voiture ↠ { marque : "Renault", couleur : "rouge", kilométrage : 45 000 }*
- *voiture2 : Voiture ↠ { marque : "Toyota", couleur : "blanche", kilométrage : 12 000 }*

Toutes partagent la même structure (définie une fois dans la classe), mais chacune a ses propres valeurs.

Classe Abstraite (Abstract Class)

Une classe **abstraite** est un **modèle incomplet** : impossible de créer une instance directe à partir d'elle. Elle existe pour factoriser ce qui est commun à plusieurs **classes enfants**, en laissant au moins une opération non implémentée.

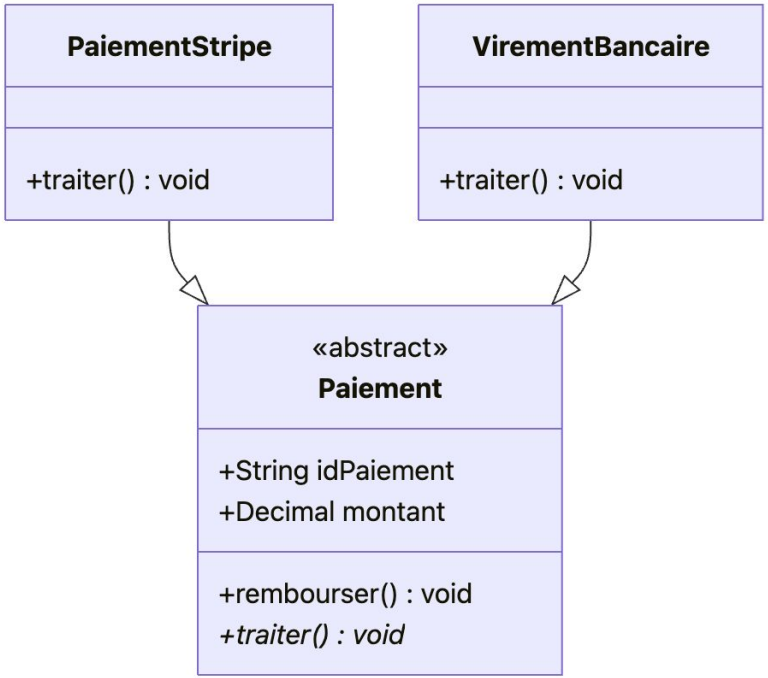
- Marquée <<abstract>> (ou nom en italique)
- Peut contenir des opérations **abstraites** (signature seule, à implémenter par les enfants) **et** des opérations **concrètes** (implémentation héritée par toutes les enfants)

Analogie

La catégorie « **Véhicule** » est trop générique pour être un objet réel — personne ne dit « j'ai acheté un Véhicule ». On achète forcément une **Voiture**, une **Moto** ou un **Camion**.

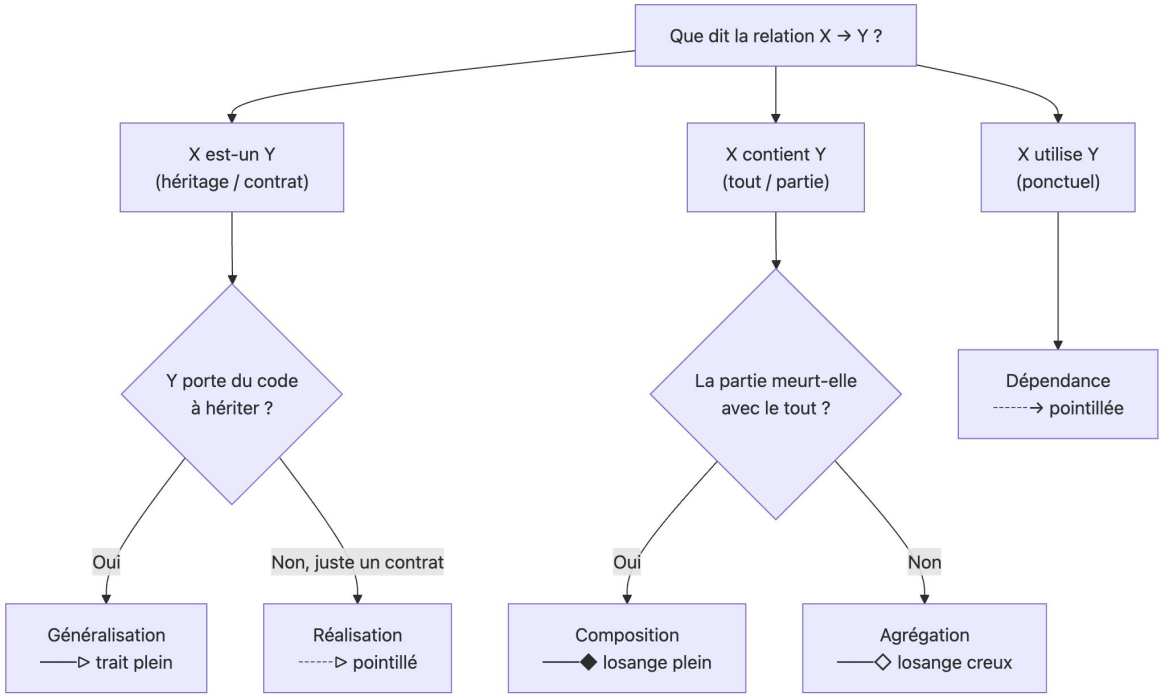
Véhicule est la classe **abstraite** ; Voiture, Moto, Camion sont les classes concrètes qu'on peut instancier. Tous les véhicules ont une marque et un kilométrage (factorisés dans Véhicule), mais chacun se déplace à sa façon (méthode démarrer() redéfinie).

Exemple



Dans NovaTrade, Paiement est abstraite : on ne peut pas instancier « un Paiement » tout court — chaque paiement est forcément un PaiementStripe ou un VirementBancaire, qui implémentent chacun leur propre traiter().

Récapitulatif — Distinguer les 6 concepts en 3 questions



Et la Classe Abstraite (<<abstract>>) — où la situer ?

Ce n'est pas une relation mais un marqueur sur la classe : « je suis un modèle incomplet, on ne peut pas créer d'instance directe de moi ». Elle joue presque toujours le rôle de parent dans une Généralisation (ex. Animal, Véhicule, Paiement) ou de classe enrichie par opposition à une simple interface (qui sert de pure Réalisation). Pensez-la comme un plan d'architecte sans permis de construire — utile comme base, jamais comme bâtiment fini.

Mémo Abstraite vs Interface : la classe abstraite peut contenir du code partagé (attributs, méthodes implémentées) ; l'interface ne contient que des signatures. En cas de doute : si les enfants ont besoin de partager du code, choisir abstraite ; sinon interface.

Pièges courants — Classes

- **Nommer une classe avec un verbe** (GestionnaireDeCommandes, TraiterFacture) — ce sont des fonctions, pas des classes. Toujours un **nom commun au singulier**.
- **Confondre Agrégation et Composition** ↠ presque toutes les relations « contient » métier sont des **compositions**.
- **Omettre la multiplicité** ↠ spécification incomplète. Encode des règles métier critiques.
- **Trop de relations** ↠ diagramme illisible. Découper par domaine fonctionnel.
- **Confondre Généralisation et Association** ↠ test « est-un » vs « a-un ».

Points clés — Classes

Une **classe** = rectangle à 3 compartiments. **PascalCase**, **nom commun au singulier**, marqueurs de visibilité (+, -, #, ~).

La **multiplicité** sur chaque extrémité encode des règles métier : nullabilité, cardinalité — impacte directement la BDD.

Agrégation (losange creux) = cycle de vie indépendant ; **Composition** (losange plein) = cycle de vie dépendant. En cas de doute ↠ composition.

Généralisation (triangle creux plein) = « est un » ; **Réalisation** (triangle creux pointillé) = « implémente » un contrat d'interface.

Le diagramme de classes est le **vocabulaire maître** — tous les autres diagrammes (Séquence, Activité, État, BDD) doivent réutiliser ses noms textuellement.

État

État — Définition

Un **diagramme d'état** (State Machine Diagram) montre comment un **objet unique évolue** au cours de sa vie — états, événements et conditions de transition.

- Pensez à un **feu de circulation** : toujours dans exactement un état (Rouge, Orange, Vert), il change selon des règles déterministes.
- Outil de référence pour modéliser les **cycles de vie** : commandes, factures, tickets, abonnements.

Définition - OMG UML 2.5

Diagramme comportemental qui modélise les **états discrets** d'un classifieur unique (typiquement une classe), les **événements** (events) qui déclenchent les **transitions** (transitions), et les **actions** (actions) exécutées à l'entrée, à la sortie ou pendant le séjour.

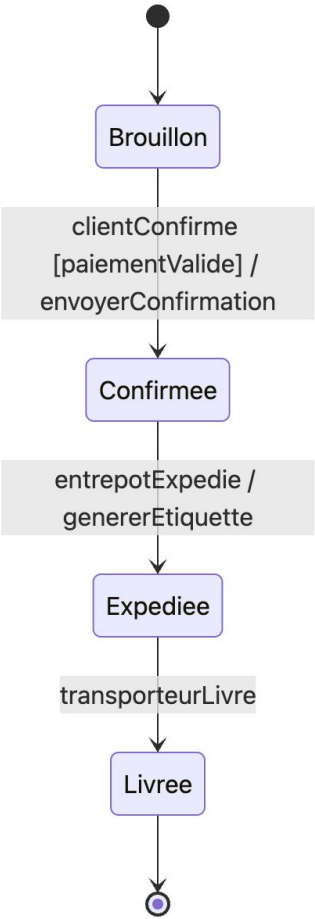
États & Transitions

État (State)

- **Rectangle aux coins arrondis** contenant le nom
- Compartiments optionnels : entry / (entrée), do / (séjour), exit / (sortie)
- **Pseudo-état initial** ● (cercle plein) et **état final** ⦿ (cible)

Transition (Transition)

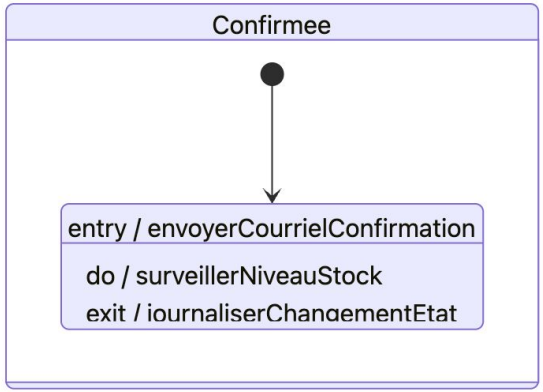
- Flèche état source ↠ état cible avec étiquette : événement [garde] / action



Actions internes — Entry, Do, Exit

Trois comportements internes qui s'exécutent **automatiquement**, sans déclencheur explicite :

- **entry** / — s'exécute **une fois à l'entrée**, avant toute autre activité
- **do** / — s'exécute **en continu pendant le séjour** ; interrompue par toute transition sortante
- **exit** / — s'exécute **une fois à la sortie**, après l'interruption du do /

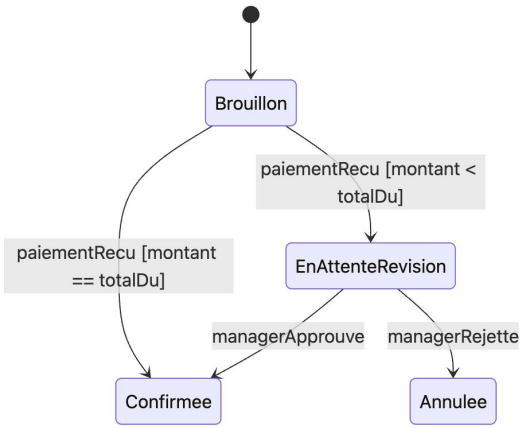


Sépare ce qui se passe à cause de l'état de ce qui se passe à cause de la transition — étiquettes plus propres.

Conditions de Garde (Guards)

Expression booléenne entre crochets [ ] sur une transition.

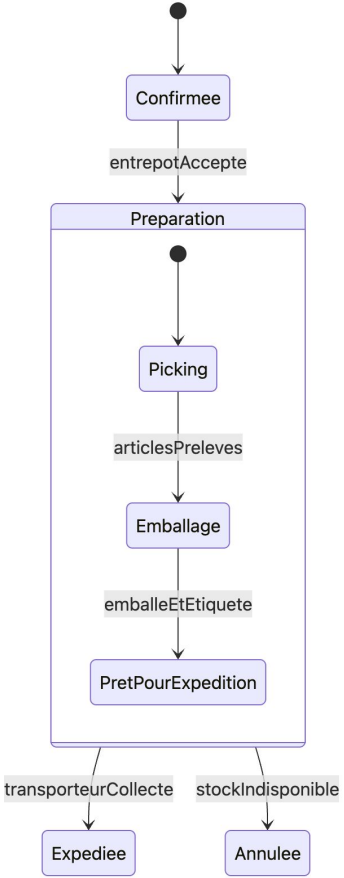
- La transition se déclenche seulement si l'événement survient et la garde est vraie.
- Permet à un même événement de mener à des états différents selon les données. Les gardes doivent être mutuellement exclusives pour chaque paire (état, événement).



États Composites (Composite States)

Un **état composite** contient sa propre **machine à états imbriquée** — équivalent UML d'un **sous-processus** (Sub-Process) BPMN.

- Toute transition quittant l'état composite (depuis sa bordure) **interrompt** immédiatement toute activité interne. Cache la complexité derrière une frontière nommée.



Pseudo-États — Historique, Choix, Jonction

Historique (History — H)

- Permet à un état composite de reprendre depuis le sous-état où il se trouvait avant interruption. H = superficiel, H* = profond.

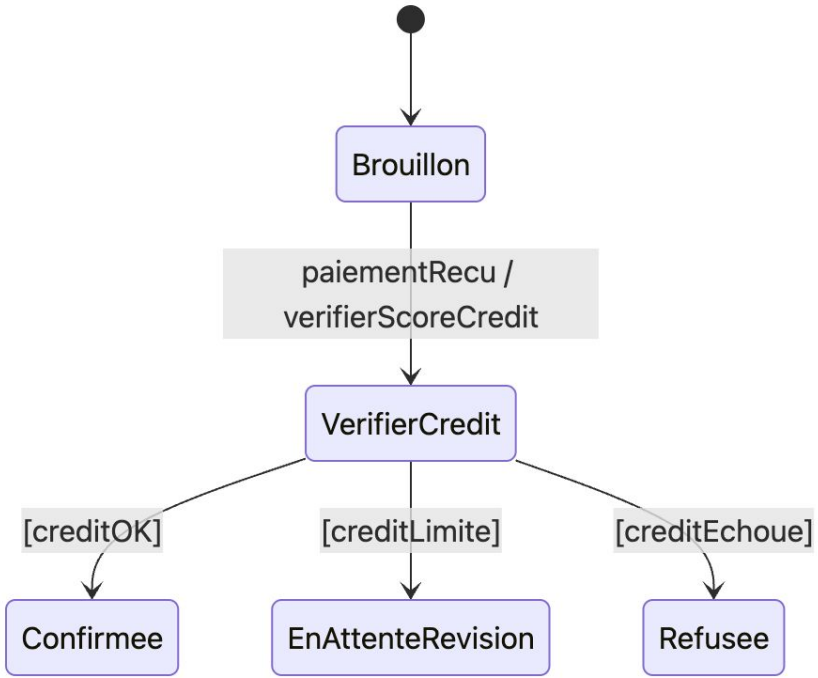
Choix (Choice — losange ◇)

- Branchement dynamique — gardes évaluées après l'action de transition. Au moins une branche [else] requise.

Jonction (Junction — cercle ○)

- Branchement statique — gardes évaluées avant la transition.

Pseudo-États — Historique, Choix, Jonction



Points clés — État

- Un **diagramme d'état** modélise le **cycle de vie d'un objet unique** — chaque état, chaque événement, chaque action.
- Une **transition** porte un triplet événement [garde] / action — chaque partie est optionnelle.
- Les actions **entry** / **do** / **exit** séparent le comportement persistant de la logique de transition.
- Les **états composites** cachent un cycle de vie imbriqué derrière une frontière, comme un sous-processus BPMN.
- **Choix** (◇) évalue les gardes **après** l'action ; **Jonction** (○) les évalue **avant**.

Séquence

Séquence — Définition

Un **diagramme de séquence** (Sequence Diagram) montre comment des objets **collaborent** pour réaliser un **scénario unique** : qui envoie quel message à qui, et dans quel ordre.

- **Axe horizontal** : les participants (lignes de vie — lifelines)
- **Axe vertical** : le temps qui s'écoule vers le bas

Outil de référence pour : **contrats d'API**, débogage inter-services, documentation des cas d'utilisation au niveau technique.

Définition - OMG UML 2.5

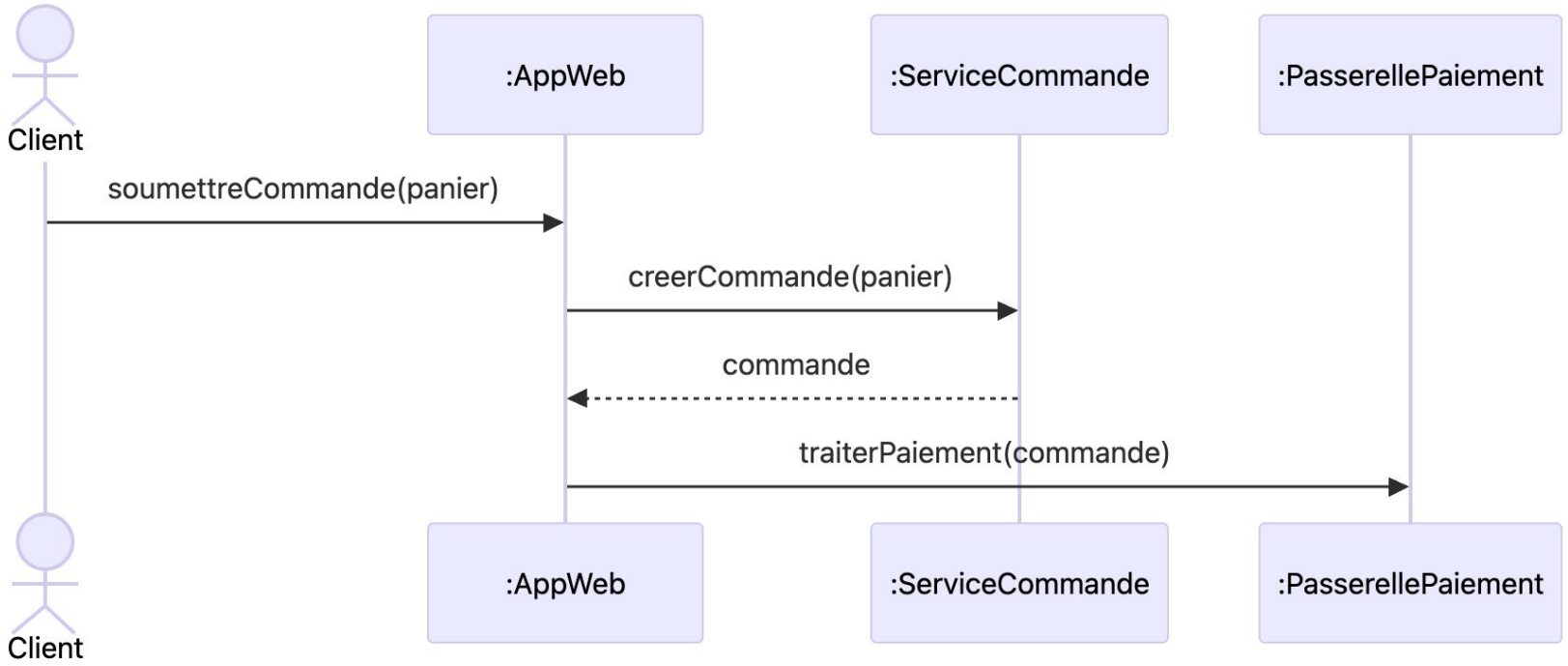
Diagramme d'interaction qui montre l'échange ordonné de messages entre un ensemble de **lignes de vie** (lifelines) participant à un seul scénario. Inclut les **spécifications d'exécution** (barres d'activation) et les **fragments combinés** (Combined Fragments) pour boucles, alternatives et exécution parallèle.

Lignes de Vie (Lifelines)

Une **ligne de vie** = un participant : instance d'objet, composant ou acteur.

- **Boîte nommée** au format nomInstance : NomClasse (ex. commande : Commande)
- **Ligne pointillée verticale** descendant tant que le participant existe
- Les **acteurs** (Actor) sont à gauche, les composants à droite

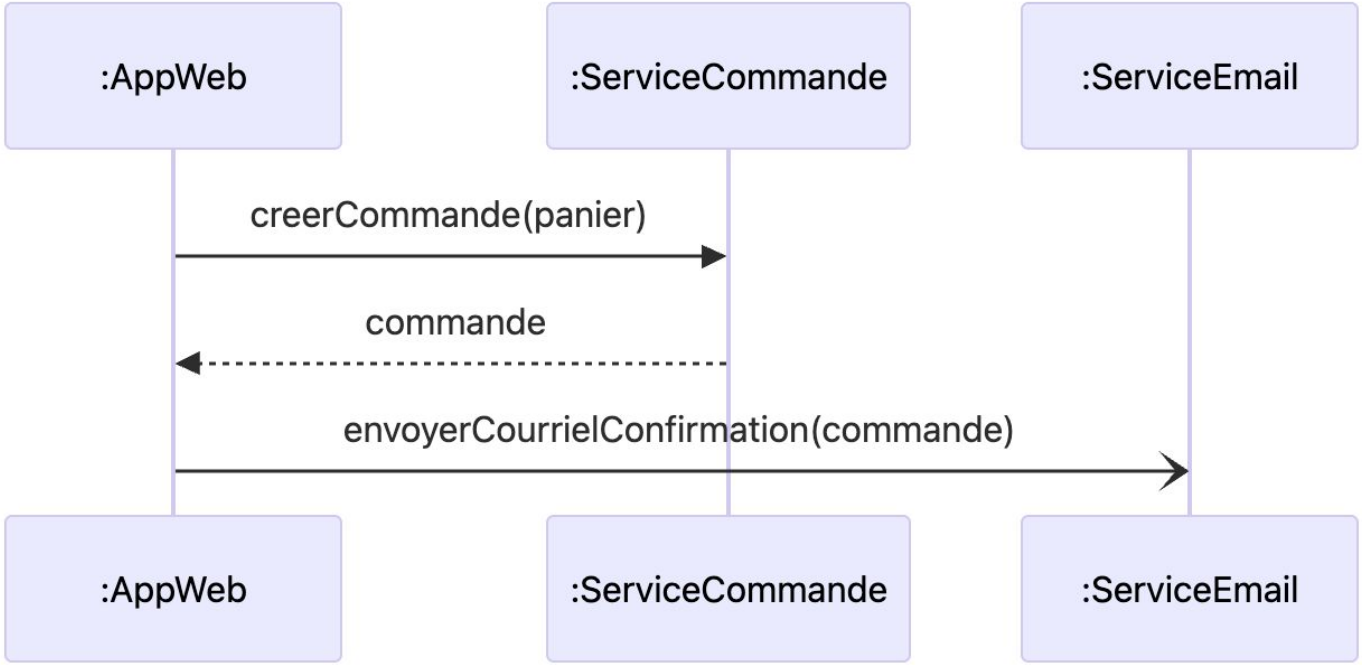
Lignes de Vie (Lifelines)



Types de Messages (Sequence)

Type	Notation	Signification
Synchrone Synchronous	——▶	L'appelant **se bloque** et attend un retour.
Asynchrone Asynchronous	——→	**Fire-and-forget** — pas d'attente de réponse.
Retour Return	- - - - - - ▶	Valeur ou contrôle renvoyé à l'appelant.
Création Create	——▶ <<create>>	Instancie un nouvel objet et sa ligne de vie.

Types de Messages (Sequence)

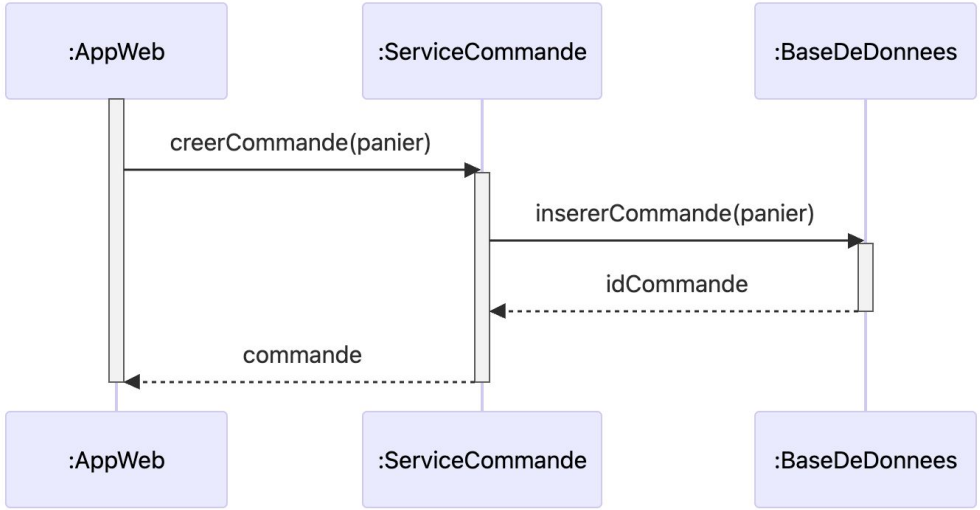


Les étiquettes portent le nom de l'opération + arguments : traiterPaiement(montant: 149.90).

Barres d'Activation (Activation Bars)

Mince **rectangle vertical** sur une ligne de vie : représente la période durant laquelle le participant **s'exécute activement** (ou attend une réponse synchrone).

- Rendent visible la **profondeur des appels** et qui « tient la main » à chaque instant.



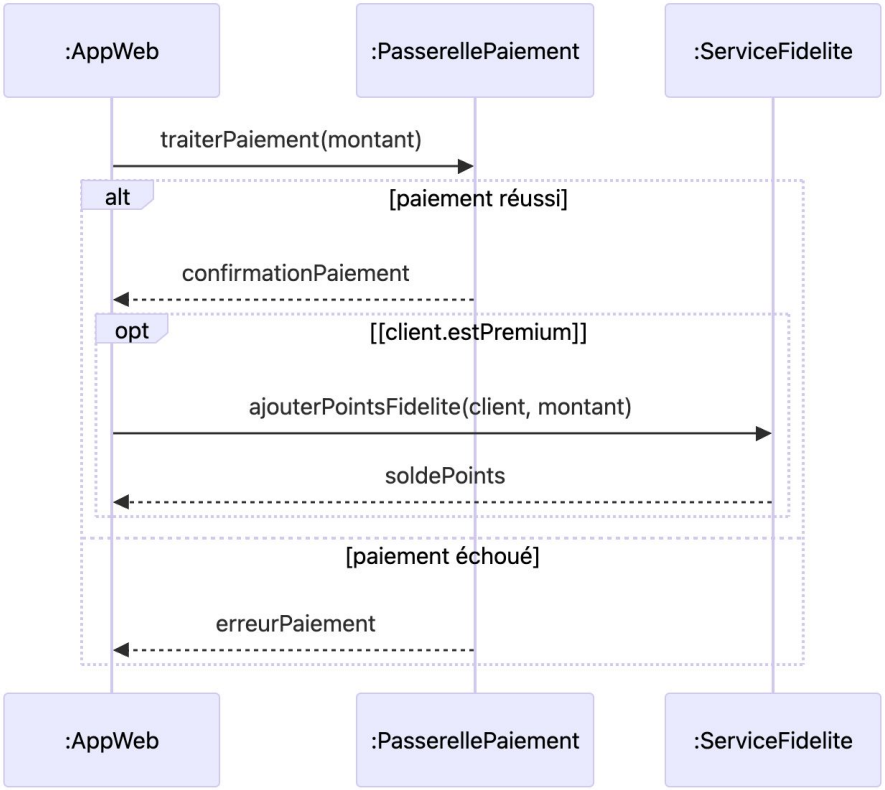
Fragments Combinés (Combined Fragments)

Cadre rectangulaire avec un **opérateur** dans le coin supérieur gauche pour exprimer la logique conditionnelle, répétée ou parallèle.

Opérateur	Signification
`alt`	**Alternative** — if/else ; un seul opérande
`opt`	**Optionnel** — comme if sans else
`loop`	**Boucle** — répète tant que la garde est vraie
`par`	**Parallèle** — tous les opérandes simultanément
`ref`	**Référence** — intègre un autre diagramme
`break`	**Rupture** — quitte la boucle englobante

*Les **gardes** s'écrivent [condition] en haut de chaque opérande. Les fragments peuvent être **imbriqués**.*

Exemple : alt + opt pour le paiement



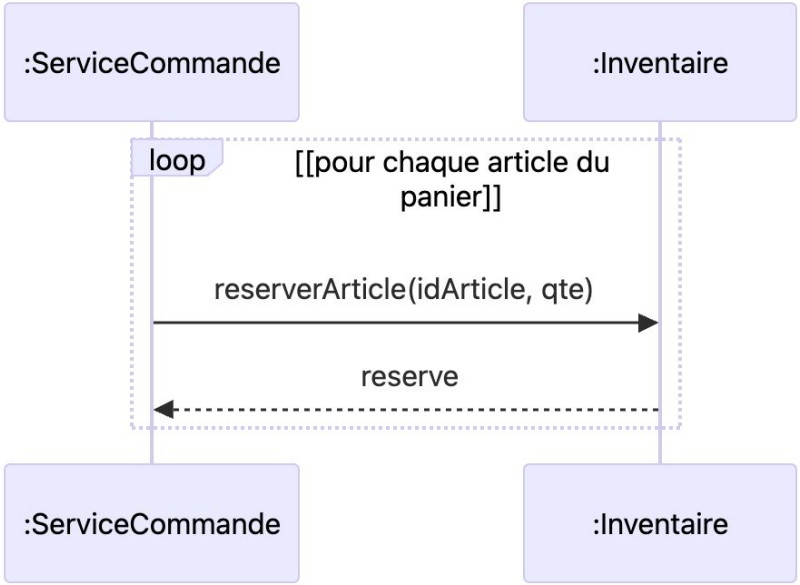
Le bloc alt divise selon le résultat ; à l'intérieur, le bloc opt ne s'exécute que si la garde [estPremium] est vraie.

Boucles & Création/Destruction

Création / Destruction

- <<create>> instancie une ligne de vie en cours de diagramme
- <<destroy>> termine la ligne de vie par un X

Un auto-message (self-message) — flèche revenant à la même ligne de vie — modélise un appel interne ou récursif.



Points clés — Séquence

- Un **diagramme de séquence** modélise **un seul scénario** : qui envoie quoi à qui, dans quel ordre — temps vers le bas, participants gauche-droite.
- **Types de messages** : synchrones (attente bloquante), asynchrones (fire-and-forget), retours, création.
- Les **barres d'activation** rendent visibles la profondeur des appels et les relations de blocage.
- **Fragments combinés** (alt, opt, loop, par, ref) apportent la logique conditionnelle sans casser la structure temporelle.
- Les diagrammes de séquence **implémentent les cas d'utilisation et complètent les diagrammes d'état** en montrant les messages qui déclenchent les transitions.

Méthode de modélisation

Le workflow en 6 étapes

01 **Identifier l'objectif**
Question ›› type de diagramme

02 **Capturer les exigences**
Cas d'Utilisation : Diagramme + descriptions textuelles

03 **Concevoir la structure**
Diagramme de Classes : Modèle de domaine validé

04 **Spécifier le comportement**
Séquence (par flux) + État (par classe à statuts)

05 **Vérifier la cohérence**
Audit inter-diagrammes : noms, scénarios, états

06 **Flux complexes**
Diagramme d'Activité où nécessaire

L'erreur UML la plus coûteuse : *Produire des diagrammes* ***isolément*** *(Classes par l'architecte, Séquence par l'analyste, État par le développeur) qui se contredisent. La puissance d'UML vient des* ***relations entre diagrammes****.*

Étape 1 — Choisir le bon diagramme

Avant de dessiner, **nommer la question** en une seule phrase.

Question	Diagramme
« Que doit faire le système ? »	**Cas d'Utilisation**
« Quels concepts existent ? »	**Classes**
« Comment ce scénario fonctionne ? »	**Séquence**
« Dans quels états cet objet peut-il être ? »	**État**
« Comment ce processus s'écoule-t-il ? »	**Activité**

*Mélanger les objectifs sur un seul diagramme produit un diagramme qui ne répond clairement à **aucune question**.*

Étape 2 — Cas d'Utilisation d'abord

Établissent le **vocabulaire et le périmètre** auxquels tous les diagrammes ultérieurs feront référence.

Diagramme de Cas d'Utilisation

5 à 10 cas par frontière de système

Descriptions Textuelles

Précondition, flux principal, alternatifs, exception, postcondition

⚠️ **Les Cas d'Utilisation ne sont pas des écrans**

« Cliquer sur Soumettre » ❌ — c'est un événement UI.
« Passer une commande » ✅ — c'est un objectif.

*Un cas d'utilisation est complet lorsque sa **postcondition** décrit un changement significatif d'état du système.*

Étape 3 — Diagramme de Classes en 3 passes

Construire en **trois passes**, en validant après chacune :

01 **Identifier les entités**
Nommer chaque concept métier (5 à 15 classes)

02 **Ajouter les relations**
Associations, multi-structures et multiplicité

03 **Détails techniques**
Attributs et opérations typés avec sens métier

💡 **Le diagramme de classes devient le vocabulaire maître :**
Chaque élément nommé sera réutilisé **textuellement** dans Séquence, État et Code.
Une discordance introduit des bogues d'implémentation.

*Une discordance (**commande.valider()** vs **commande.verifierValidite()**) est une source majeure de bogues.*

Étape 4 — Séquence & État après les Classes

L'ordre correct est **non négociable** :

01 **Cas d'Utilisation**
Établit les scénarios

02 **Classes**
Établit le vocabulaire

03 **Séquence**
Spécifie un scénario (classes/opérations)

04 **État**
Cycle de vie d'une classe à statuts

📋 **Règles pratiques :**

- 1 **Séquence** par flux principal + 1 par flux alternatif significatif.
- 1 **État** par classe avec un attribut 'statut' à plus de **2 valeurs**.

*Tenter d'écrire un diagramme de séquence avant les classes = **noms inventés** qui divergent du modèle de domaine.*

Étape 5 — Vérifier la cohérence

Quatre correspondances à contrôler :

Vocabulaire

Chaque nom dans Séquence/État correspond-il **exactement** à Classes ?

Couverture des scénarios

Chaque cas d'utilisation a-t-il **au moins un** Séquence ou Activité ?

Atteignabilité des états

Chaque état est-il **atteignable** ? Chaque transition est **déclenchée** par un message Séquence.

Alignement du cycle de vie

Les opérations des Classes correspondent-elles aux transitions de l'État ?

*Effectuer un **audit des noms** : extraire tous les noms de tous les diagrammes, comparer au diagramme de classes.*

Étape 6 — Activité pour les flux complexes

Quand utiliser **Séquence** vs **Activité** ?

Séquence

Scénarios pilotés par l'**échange de messages** entre objets identifiables.

Activité

Scénarios pilotés par la **logique de flux de contrôle** (branchements complexes, parallélisme).

⚠️ **Attention à la lisibilité**

Les `alt`/`loop`/`par` fortement imbriqués dans un Séquence deviennent illisibles → passer à **Activité**.

*Souvent **les deux sont nécessaires** : Activité pour le flux logique (audience métier), Séquence pour le protocole technique (audience développeurs).*

Bonnes pratiques de modélisation

- **Une question, un diagramme** — formuler la question avant de dessiner
- **Vocabulaire cohérent** — Classes est la source de vérité ; tout le reste s'y réfère
- **Un scénario par Séquence, une classe par État** — diagrammes ciblés > diagrammes fourre-tout
- **Multiplicité partout** — chaque association porte cardinalité aux deux extrémités
- **Détail adapté à l'audience** — analyste métier ≠ développeur ; deux versions si nécessaire
- **Liens explicites entre diagrammes** — «implements UC-01», «lifecycle of Commande»
- **Validation contre le code** ou contre l'expert du domaine
- **Modèle vivant** — propriétaire nommé, version, date de dernière revue

Points clés — Méthode

- **Nommer la question** avant de dessiner — un type de diagramme par question.
- **Cas d'Utilisation d'abord** pour fixer vocabulaire et périmètre.
- **Classes ensuite** — vocabulaire maître que les Séquence/État doivent référencer textuellement.
- **1 Séquence par flux, 1 État par classe à statuts** — uniquement après stabilisation des Classes.
- **Vérifier la cohérence** : audit des noms, couverture, atteignabilité, alignement cycle de vie.
- **Modèle vivant** — propriétaire, version, date de revue.

Ce que vous savez maintenant lire

Avec UML, vous pouvez :

- **Définir le périmètre** d'un système (Cas d'Utilisation)
- **Modéliser le domaine métier** (Classes : entités, attributs, relations)
- **Spécifier le cycle de vie** d'un objet (État)
- **Concevoir les contrats d'API** (Séquence)
- **Documenter les opérations système** (Activité)
- **Communiquer une architecture** sans plonger dans le code

L'essentiel à retenir

Un modèle UML n'a pas pour but d'être joli — il a pour but de produire une **spécification partagée et non ambiguë** entre analystes, architectes, développeurs et testeurs.

Un modèle est correct **lorsque chaque diagramme répond à une question spécifique, que chaque concept est cohérent à travers les diagrammes**, et **que toute partie prenante peut le lire et comprendre** ce que le système fait et comment il est construit.